

Python Funktionen

Funktionen definieren und aufrufen

Funktionsdefinition

Eine Funktion wird mit dem Schlüsselwort `def` definiert und besteht aus:

- **Header:** Erste Zeile mit dem Namen der Funktion und den Parametern in Klammern
- **Body:** Der eingerückte Code-Block (üblicherweise 4 Leerzeichen), der bei Funktionsaufruf ausgeführt wird

```
def funktionsname(parameter1, parameter2, ...):  
    # Funktionskörper  
    anweisung1  
    anweisung2  
    ...
```

Beispiel einer einfachen Funktion

```
def print_lyrics():  
    print("I'm a lumberjack, and I'm okay.")  
    print("I sleep all night and I work all day.")
```

Funktionsaufruf

Eine Funktion wird aufgerufen, indem man ihren Namen gefolgt von runden Klammern schreibt:

```
print_lyrics() # Ruft die Funktion auf und führt ihre Anweisungen aus
```

Funktionen mit Parametern

Parameter sind Variablen, die in der Funktionsdefinition deklariert und bei jedem Aufruf mit konkreten Werten (Argumenten) gefüllt werden:

```
def print_twice(string):  
    print(string)  
    print(string)  
  
# Aufruf mit einem String-Argument  
print_twice('Dennis Moore, ')  
  
# Aufruf mit einer Variable als Argument
```

```
line = 'Dennis Moore, '  
print_twice(line)
```

Mehrere Parameter

Funktionen können mehrere Parameter haben:

```
def repeat(word, n):  
    print(word * n)  
  
# Aufruf mit zwei Argumenten  
spam = 'Spam, '  
repeat(spam, 4) # Gibt "Spam, Spam, Spam, Spam, " aus
```

Funktionskomposition

Funktionen rufen andere Funktionen auf

Funktionen können andere Funktionen aufrufen, was die Wiederverwendung von Code ermöglicht:

```
def first_two_lines():  
    repeat(spam, 4)  
    repeat(spam, 4)  
  
def last_three_lines():  
    repeat(spam, 2)  
    print('(Lovely Spam, Wonderful Spam!)')  
    repeat(spam, 2)  
  
def print_verse():  
    first_two_lines()  
    last_three_lines()
```

Aufrufhierarchie

- Beim Aufruf einer Funktion wird deren Code ausgeführt
- Wenn eine Funktion eine andere aufruft, wird die Ausführung der ersten Funktion pausiert
- Nach Beendigung der aufgerufenen Funktion wird die Ausführung der ersten Funktion fortgesetzt

Wiederholung mit for-Schleifen

Grundstruktur einer for-Schleife

```
for variablenname in range(anzahl):  
    # Schleifenkörper  
    anweisung1
```

```
anweisung2
...
```

Beispiel

```
for i in range(2):
    print(i) # Gibt "0" und dann "1" aus
```

Mit Funktionen kombinieren

```
def print_n_verses(n):
    for i in range(n):
        print_verse()
        print() # Leere Zeile zwischen den Versen
```

Wirkungsweise von range

- `range(2)` erzeugt die Sequenz: 0, 1
- `range(1, 5)` erzeugt die Sequenz: 1, 2, 3, 4
- `range(0, 10, 2)` erzeugt die Sequenz: 0, 2, 4, 6, 8
- `range(5, 0, -1)` erzeugt die Sequenz: 5, 4, 3, 2, 1

Variablen-Geltungsbereich (Scope)

Lokale Variablen

- Variablen, die innerhalb einer Funktion definiert werden, sind lokal zur Funktion
- Sie existieren nur während der Ausführung der Funktion
- Nach dem Ende der Funktion werden lokale Variablen gelöscht

```
def cat_twice(part1, part2):
    cat = part1 + part2 # Lokale Variable 'cat'
    print_twice(cat)

# cat existiert hier nicht mehr
# print(cat) # Würde einen NameError verursachen
```

Parameter sind lokal

Parameter funktionieren wie lokale Variablen und existieren nur innerhalb der Funktion:

```
def print_twice(string): # 'string' ist ein lokaler Parameter
    print(string)
```

```
print(string)

# 'string' existiert hier nicht
```

Globale Variablen

Variablen, die außerhalb von Funktionen definiert werden, sind global und können innerhalb von Funktionen verwendet werden:

```
message = "Hello, World!" # Globale Variable

def greet():
    print(message) # Verwendet die globale Variable 'message'

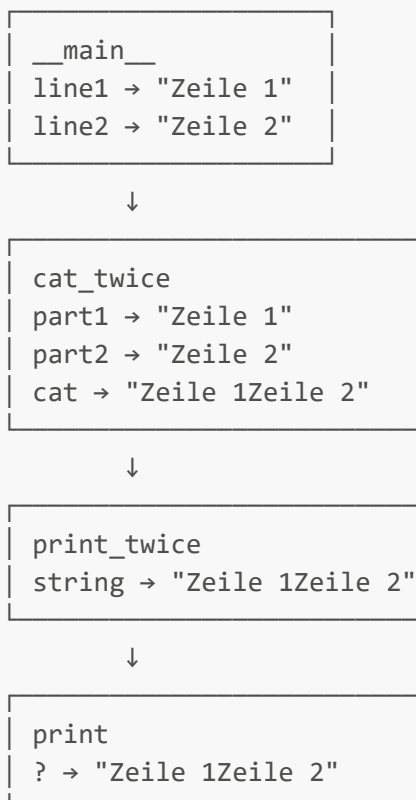
greet() # Gibt "Hello, World!" aus
```

Stack-Diagramme und Debugging

Stack-Diagramm

- Visuelle Darstellung der Funktionsaufrufe und ihrer Variablen
- Jede Funktion wird durch einen Frame (Kasten) dargestellt
- Der Frame enthält Parameter und lokale Variablen der Funktion

Beispiel:



Traceback

- Bei Laufzeitfehlern zeigt Python einen Traceback an
- Ein Traceback listet die aufgerufenen Funktionen in umgekehrter Reihenfolge auf
- Hilft bei der Identifikation, wo der Fehler aufgetreten ist

Beispiel für einen Traceback:

```
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
  File "<stdin>", line 3, in cat_twice
  File "<stdin>", line 2, in print_twice
NameError: name 'cat' is not defined
```

Typische Fehler in Funktionen

NameError

- Entsteht, wenn auf eine Variable zugegriffen wird, die nicht definiert ist
- Häufig durch Tippfehler oder Zugriff auf lokale Variablen außerhalb ihres Geltungsbereichs

```
def print_twice(string):
    print(cat) # NameError: cat ist nicht definiert
    print(cat)
```

TypeError

- Entsteht, wenn eine Funktion mit falschen Argumenttypen aufgerufen wird
- Oder wenn die Anzahl der Argumente nicht mit der Anzahl der Parameter übereinstimmt

```
def multiply(a, b):
    return a * b

multiply(3) # TypeError: fehlendes Argument
multiply('text', 5.0, True) # TypeError: zu viele Argumente
```

SyntaxError

- Entsteht bei Fehlern in der Struktur des Codes
- Zum Beispiel fehlende Doppelpunkte, falsche Einrückung oder fehlende Klammern

```
def broken_function() # SyntaxError: fehlendes ':'
    print("Dies wird nie ausgeführt")
```

Warum Funktionen?

Vorteile von Funktionen

1. **Wiederverwendbarkeit:** Code muss nur einmal geschrieben werden
2. **Abstraktionsniveau:** Komplexe Operationen können einen einfachen Namen erhalten
3. **Modularität:** Programme können in kleinere, überschaubare Teile aufgeteilt werden
4. **Testbarkeit:** Einzelne Funktionen können unabhängig getestet werden
5. **Wartbarkeit:** Änderungen müssen nur an einer Stelle vorgenommen werden

Best Practices

- Eine Funktion sollte eine klar definierte Aufgabe erfüllen
- Wähle aussagekräftige Namen für Funktionen und Parameter
- Kommentiere komplexe Funktionen, um ihren Zweck zu erklären
- Vermeide zu lange Funktionen; teile sie in kleinere Funktionen auf

Übungsbeispiele

Beispiel 1: Text rechtsbündig ausgeben

```
def print_right(text):  
    # Berechne die Anzahl der benötigten Leerzeichen  
    spaces = 40 - len(text)  
    # Verwende den String-Multiplikationsoperator für Leerzeichen  
    print(' ' * spaces + text)  
  
print_right("Monty")           # Ausgabe: "                Monty"  
print_right("Python's")       # Ausgabe: "           Python's"  
print_right("Flying Circus")  # Ausgabe: "        Flying Circus"
```

Beispiel 2: Dreieck aus Zeichen erstellen

```
def triangle(char, height):  
    for i in range(height + 1):  
        print(char * i)  
  
triangle('L', 5)  
# Ausgabe:  
#  
# L  
# LL  
# LLL  
# LLLL  
# LLLLL
```

Beispiel 3: Rechteck aus Zeichen erstellen

```
def rectangle(char, width, height):
    for i in range(height):
        print(char * width)

rectangle('H', 5, 4)

# Ausgabe:
# HHHHH
# HHHHH
# HHHHH
# HHHHH
```

Beispiel 4: "99 Bottles of Beer"-Song

```
def bottle_verse(n):
    print(f"{n} bottles of beer on the wall")
    print(f"{n} bottles of beer ")
    print("Take one down, pass it around")
    print(f"{n-1} bottles of beer on the wall")

# Eine Strophe ausgeben
bottle_verse(99)

# Alle Strophen ausgeben
for n in range(99, 0, -1):
    bottle_verse(n)
    print() # Leere Zeile zwischen den Strophen
```

Glossar

Begriff	Beschreibung
Funktionsdefinition	Eine Anweisung, die eine Funktion erstellt (<code>def name():</code>)
Header	Die erste Zeile einer Funktionsdefinition
Body	Die Folge der Anweisungen innerhalb einer Funktionsdefinition
Funktionsobjekt	Ein Wert, der von einer Funktionsdefinition erzeugt wird
Parameter	Ein Name innerhalb einer Funktion, der auf einen übergebenen Wert verweist
Argument	Ein Wert, der einer Funktion beim Aufruf übergeben wird
Schleife	Eine Anweisung, die andere Anweisungen wiederholt ausführt
Lokale Variable	Eine innerhalb einer Funktion definierte Variable, nur dort verfügbar
Stack-Diagramm	Grafische Darstellung der Funktionsaufrufe und ihrer Variablen

Begriff	Beschreibung
Frame	Ein Kasten im Stack-Diagramm, der für einen Funktionsaufruf steht
Traceback	Liste der aufgerufenen Funktionen, die ausgegeben wird, wenn ein Fehler auftritt